

PROJECT & OPERATIONS DOCUMENTATION

calcu

Distributed Scientific & Financial Calculator

Microservices · Docker · NGINX · CI/CD · AWS EC2 · Kubernetes

Two guides in one document:

Part I — Complete Project & DevOps Explanation (beginner-friendly)

Part II — Manual DevOps Runbook (“What You Should Do”)

Contents

Part I — Complete Project & DevOps Explanation

1. What the application actually is
2. The folder map
3. Containers & Dockerfiles
4. NGINX — the reverse proxy / gateway
5. The CI/CD pipeline
6. Docker Compose — one file, the whole stack
7. AWS EC2 deployment — how the pieces fit
8. Kubernetes — the same app, orchestrated
9. Correctness fixes made during cleanup

Part II — Manual DevOps Runbook

- A. Things only a human can do (AWS, accounts, IAM)
 - B. Run it locally from scratch
 - C. Deploy on AWS EC2 from scratch
 - D. Secrets & environment variables
 - E. Day-2 operations
 - F. Teardown
- Appendix — one-page cheat sheet

Part I — Complete Project & DevOps Explanation

A plain-English tour of how this project is built and shipped. No prior DevOps experience assumed.

1. What the application actually is

`calcu` is a calculator deliberately split into **several small programs that talk to each other over the network** instead of one big program. Each small program is a **microservice**.

Microservice	Language	What it does	Talks to
frontend	HTML/CSS/JS	The web page you see — buttons and results.	The browser calls the other services <i>through NGINX</i> .
scientific-engine	Python (Flask)	<code>sin</code> , <code>cos</code> , <code>log</code> , <code>sqrt</code> , <code>factorial</code> , ...	Calls history-service to log each calculation.
financial-engine	Python (Flask)	Simple interest, compound interest, EMI.	Calls history-service to log each calculation.
history-service	Python (Flask + SQLAlchemy)	Reads/writes the list of past calculations.	Reads/writes PostgreSQL .
PostgreSQL	—	The database. Stores history rows on disk.	Only history-service.
nginx	—	The “front door”. Decides which service each request goes to.	All of the above.

Why split it up?

- **Independent scaling** — trig math is CPU-heavy; run 8 copies of scientific-engine and only 1 database.
- **Independent deploys** — fix a financial formula without touching anything else.
- **Clear boundaries** — only history-service knows how the database works.
- **It is a teaching project** — the smallest realistic setup that exercises every common DevOps tool.

How a single click flows through the system

```
You click "Calculate" (sqrt of 16)
|
v
Browser sends: POST /api/scientific {"operation":"sqrt","value":16}
|
v
NGINX sees the path starts with /api/scientific -> forwards to scientific-engine:5001
|
v
scientific-engine computes 4.0
|      \--> fires POST /api/history {"expression":"sqrt(16.0)","result":4.0,...}
|              |
|              v
|              history-service -> INSERT INTO calculation_history ... -> PostgreSQL
|
v
scientific-engine replies {"operation":"sqrt","value":16.0,"result":4.0}
|
v
NGINX passes it straight back to the browser -> "sqrt(16) = 4.0"
```

Every few seconds the frontend also calls `GET /api/history` so the “Live Calculation History” panel updates even when *someone else* did the maths.

2. The folder map

```
calcu/
|-- frontend/                The web UI.
|   |-- index.html           Page structure.
|   |-- style.css            Looks.
|   |-- app.js               Behaviour - the fetch() calls to /api/*.
|   \-- Dockerfile           How to package it into a container.
|
|-- services/                The three Python microservices. Same shape each:
|   |-- <service>/app.py      The Flask application (routes + logic).
|   |-- <service>/wsgi.py      Entry point Gunicorn imports in production.
|   |-- <service>/requirements.txt Python libraries to install.
|   |-- <service>/test_app.py  Unit tests (pytest).
|   |-- <service>/Dockerfile   How to package it.
|   \-- <service>/dockerignore Files to keep OUT of the image.
|
|-- db/init.sql               Creates the history table when Postgres first starts.
|-- nginx/
|   |-- nginx.conf            The routing rules (which path -> which service).
|   \-- Dockerfile            Packages nginx.conf into the calcu-nginx image.
|
|-- deploy/                  Operator tooling for servers:
|   |-- ec2-provision.sh       Installs Docker on a fresh EC2 box (run once).
|   |-- deploy.sh              Builds + starts/updates the stack.
|   \-- calcu.service          systemd unit so the stack starts on boot.
|
|-- docker-compose.yml        Describes all 6 containers + how they connect.
|-- docker-compose.override.yml Local-dev-only extras (auto-loaded).
|-- docker-compose.prod.yml   A thin overlay with EC2-only changes.
|-- .env.example               Template for secrets/config -> copy to .env.
|
|-- .github/workflows/ci.yml   GitHub Actions: test, build, scan, push.
|-- Jenkinsfile                Jenkins: roll the new images out to Kubernetes.
\-- k8s/                       Kubernetes manifests (the alternative to Compose).
```

3. Containers & Dockerfiles

The idea

A **container** is your program plus *everything it needs to run* (the right Python version, libraries, OS bits) sealed into one bundle called an **image**. “Works on my machine” stops being a problem because the machine comes *with* the program.

- **Image** = the frozen bundle (a template). Built once.
- **Container** = a running copy of an image. You can start many from one image.

A Dockerfile is the recipe for an image

services/scientific-engine/Dockerfile , annotated:

```
# ---- Build stage: install dependencies in a throwaway layer ----
FROM python:3.11-slim AS builder          # start from a small Python 3.11 OS
WORKDIR /app                             # work in /app inside the image
RUN python -m venv /opt/venv              # make a clean virtual environment
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .                   # copy ONLY the deps list first...
RUN pip install --no-cache-dir -r requirements.txt # ...so this layer is cached
                                                # unless deps change

# ---- Runtime stage: copy just what's needed to RUN ----
FROM python:3.11-slim AS runtime
RUN apt-get update && apt-get upgrade -y # patch OS security updates
  && useradd --uid 1000 appuser           # create a non-root user
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv # bring the installed libraries over
COPY app.py wsgi.py ./                  # bring the code
USER appuser                            # run as non-root (safer)
EXPOSE 5001                             # documents the port it listens on
HEALTHCHECK ... CMD curl .../health      # Docker keeps checking it's alive
CMD ["gunicorn", "--bind", "0.0.0.0:5001", "--workers", "2", "wsgi:app"]
```

Technique	Why
Multi-stage build (<code>builder</code> → <code>runtime</code>)	Compilers and build tools stay in the builder; the final image is small and has less to attack.
Copy <code>requirements.txt</code> before the code	Docker caches layers. Code changes far more often than dependencies, so most rebuilds skip re-installing everything.
USER <code>appuser</code> (non-root)	If someone breaks into the container, they are not root.
<code>apt-get upgrade</code> in the runtime stage	Picks up OS security patches released after the base image was published.
HEALTHCHECK	Docker/Kubernetes can tell a <i>hung</i> container from a <i>healthy</i> one and restart it.
<code>.dockerignore</code>	Keeps <code>test_app.py</code> , <code>__pycache__</code> , <code>.env</code> out of the image — smaller and no secrets.
Gunicorn, not <code>flask run</code>	Flask's built-in server is single-threaded and “not for production”. Gunicorn runs multiple worker processes.

The two NGINX images

`calcu-frontend` and `calcu-nginx` are **both** based on `nginx:alpine` , but:

- `calcu-frontend` copies the HTML/CSS/JS into `/usr/share/nginx/html` and just serves static files.
- `calcu-nginx` copies `nginx/nginx.conf` and acts as the **gateway** — it does not serve files, it *forwards* requests to other services.

4. NGINX — the reverse proxy / gateway

What “reverse proxy” means

A normal proxy sits in front of *you* and fetches web pages on your behalf. A **reverse proxy** sits in front of *the servers* and decides which one answers each incoming request. The client only ever sees one address.

Why calcu needs it

The browser calls `/api/scientific`, `/api/financial`, `/api/history`, and also loads `/`, `/style.css`, `/app.js`. Those live on **four different services** on four ports. Without a gateway the frontend JavaScript would need every service’s address, and browsers would block the cross-origin calls anyway. NGINX gives everything **one origin** (`http://localhost:8080`, or your EC2 address) and routes internally.

The routing rules (`nginx/nginx.conf`), explained

```
upstream scientific_upstream { server scientific-engine:5001; } # a friendly name
...

server {
    listen 80; # accept HTTP on port 80 inside the container
    client_max_body_size 1m; # reject bodies bigger than 1 MB

    # If a backend is down or a path is wrong, return JSON - not NGINX's HTML
    # error page - so the frontend's JSON.parse() never chokes.
    error_page 404 = @json_404;
    error_page 500 502 503 504 = @json_5xx;

    location /api/scientific { # path starts with this ->
        proxy_pass http://scientific_upstream/api/scientific; # send to that service
        proxy_set_header X-Real-IP $remote_addr; # tell the backend who called
    }
    location /api/financial { proxy_pass http://financial_upstream/api/financial; }
    location /api/history { proxy_pass http://history_upstream/api/history; }

    location / { # everything else ->
        proxy_pass http://frontend_upstream/; # the static frontend
    }
}
```

`scientific-engine`, `financial-engine`, etc. are **DNS names** that Docker Compose (and Kubernetes) create automatically — every service can reach every other by its name on the private network.

On Kubernetes

Same job, two options: `k8s/gateway.yaml` runs the *identical* `calcu-nginx` image as a pod; `k8s/ingress.yaml` hands the routing to the cluster’s shared “NGINX Ingress Controller” instead.

5. The CI/CD pipeline

CI (Continuous Integration) = every code change is automatically tested and built. **CD (Continuous Deployment)** = those built artifacts are automatically rolled out. This project splits them across two tools on purpose (both are

common in industry).

```
+----- GitHub Actions (CI) -----+
| git push -> TEST -> BUILD images -> SCAN (Trivy) -> PUSH |
|           (pytest)  (docker build)  (vuln check)   Docker Hub|
+-----+-----+
|                               |
|           images now exist at |
|           docker.io/<you>/calcu-<service>:<git-sha> |
|                               |
|                               v                               |
+----- Jenkins (CD) -----+
| trigger -> connect to Kubernetes -> kubectl set image (rolling) |
|           -> wait for healthy rollout -> Slack OK (or rollback) |
+-----+-----+
```

CI stages — .github/workflows/ci.yml

Stage	Job	What happens	Fails the build if...
Test	test	3 parallel runners (one per Python service), install deps, run <code>pytest -v</code> .	any test fails
Build	build-scan-push	<code>docker build</code> for all 5 images, tagged with the git commit SHA and <code>latest</code> .	a Dockerfile is broken
Scan	(same job)	Trivy checks each image for CRITICAL/HIGH CVEs. Non-blocking (<code>--exit-code 0</code>) — reports but does not stop the pipeline.	never (flip to <code>--exit-code 1</code> for a hard gate)
Push	(same job)	<code>docker push</code> both tags to Docker Hub. Only on a push to <code>main</code> , never on a PR.	Docker Hub rejects the push
Notify	notify	One Slack message: SUCCESS / FAILURE with repo, branch, commit, author. Runs even if earlier stages failed.	never

Why tag with the git SHA? So Jenkins can later deploy *one exact, known-good build* — not a moving `latest`.

About Trivy and the pinned version. Trivy is a free vulnerability scanner. In March 2026 the `aquasecurity/trivy-action` Marketplace action *and* some Trivy releases were briefly compromised in a supply-chain attack. This workflow therefore runs Trivy from its **official container image at a fixed pre-incident tag** (`aquasec/trivy:0.55.2`) rather than the Marketplace action — a small, deliberate example of supply-chain hardening.

CD stages — Jenkinsfile

Stage	What happens
Checkout Code	Pull the repo; work out which image tag to deploy (build parameter, or the triggering commit SHA, or <code>latest</code>).
Notify: Start	Post “Deployment STARTED” to Slack.
Connect to Kubernetes	Load the kubeconfig from Jenkins’ credential store; <code>kubectl get nodes</code> to prove the connection.
Deploy: Rolling Update	For each service: <code>kubectl set image deployment/<svc> <svc>=<you>/calcu-<svc>:<tag></code> . Kubernetes starts new pods and retires old ones one at a time — no downtime.
Verify Rollout Health	<code>kubectl rollout status</code> waits until every new pod passes its readiness probe; errors if one does not within 180s.
<code>post { success }</code>	Slack success.
<code>post { failure }</code>	Slack failure, then <code>kubectl rollout undo</code> on every deployment (automatic rollback), then Slack “rollback completed”.

Why secrets are never in these files

`ci.yml` and `Jenkinsfile` live in Git. Anyone who can read the repo could read a hard-coded password or webhook URL. Both files only *reference* secrets by name (`${ secrets.DOCKERHUB_TOKEN }` , `credentials('slack-webhook-url')`); you paste the real values into GitHub’s and Jenkins’ own encrypted stores (see Part II §D).

6. Docker Compose — one file, the whole stack

`docker-compose.yml` is a single description of all six containers: which image each uses, what environment variables it gets, which depends on which, and what network they share.

```

name: calcul # prefixes the project's resources

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-calcul_pass} # from .env, default calcul_pass
    volumes:
      - calcul-postgres-data:/var/lib/postgresql/data # data survives restarts
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro # runs on first start
    healthcheck: # "is Postgres accepting connections?"
      test: ["CMD-SHELL", "pg_isready -U ... -d ..."]

  history-service:
    build: { context: ./services/history-service } # build from that Dockerfile
    image: calcul-history-service:${IMAGE_TAG:-latest} # ...and name the result this
    environment:
      DATABASE_URL: postgresql+psycopg2://.../calcul_db # how to reach Postgres
    depends_on:
      postgres: { condition: service_healthy } # wait for the healthcheck

  nginx:
    build: { context: ./nginx }
    ports:
      - "${HTTP_PORT:-8080}:80" # host:container - the ONLY port exposed to you

networks:
  calcul-net: { driver: bridge } # private network; service names resolve on it

volumes:
  calcul-postgres-data: # named volume = the database's disk

```

Term	Meaning
service	One container definition.
build vs image	<code>build</code> says <i>how</i> to make the image; <code>image</code> <i>names</i> it. Both = <code>docker compose build</code> produces a predictably-named image.
depends_on + condition: service_healthy	Start ordering. <code>history-service</code> will not start until <code>Postgres</code> passes its healthcheck.
volume	Disk storage that outlives the container. Without it, <code>docker compose down</code> would delete all history.
network	The private LAN. <code>history-service</code> can reach <code>postgres</code> because they share <code>calcul-net</code> .
\${VAR:-default}	Read <code>VAR</code> from <code>.env</code> ; use <code>default</code> if unset. This makes the <i>same file</i> work locally and on EC2.
expose vs ports	<code>expose</code> = reachable by other containers only. <code>ports</code> = published to the host. Only NGINX publishes a port.

The overlay files

- `docker-compose.override.yml` — auto-loaded on a plain `docker compose up` (local dev only). Publishes Postgres on `127.0.0.1:5432` for your `psql` /GUI. The EC2 path uses explicit `-f` flags, so it never picks this up.
- `docker-compose.prod.yml` — EC2 changes: `restart: always` (survive reboot) and per-container log rotation (3×10 MB) so a small instance disk never fills. `HTTP_PORT=80` in `.env` makes the site answer on the normal web port.

7. AWS EC2 deployment — how the pieces fit

```
Internet
| HTTP :80
v
+-----+
| AWS EC2 instance (Ubuntu) |
|                             |
| Security Group -- allows :22 (you), :80 |
|                             |
| Docker Engine               |
| \- docker compose project "calcu" |
|     calcul-nginx :80 +- calcul-frontend |
|                     +- calcul-scientific |
|                     +- calcul-financial |
|                     \- calcul-history - calcul-postgres (private, volume-backed) |
|                             |
| systemd unit "calcu.service" -> starts it on boot |
+-----+
```

There is **no AWS-specific code** in the app. EC2 is just “a Linux box with a public IP”. The only AWS things a human configures are the instance, its **security group** (a cloud firewall — you must open port 80), and optionally an **IAM role** if you later want the instance to pull private images from ECR or read secrets from Parameter Store.

File	Run it...	Purpose
<code>deploy/ec2-provision.sh</code>	once, on a fresh instance	Installs Docker + Compose from Docker’s official apt repo, adds you to the <code>docker</code> group, turns on the host firewall (<code>ufw</code>).
<code>deploy/deploy.sh</code>	every deploy	<code>git pull</code> → <code>docker compose build</code> → <code>up -d</code> . <code>ENV=prod</code> adds the overlay. Idempotent.
<code>deploy/calcu.service</code>	install once	systemd unit; <code>systemctl enable</code> it so the stack starts automatically after a reboot.

8. Kubernetes — the same app, orchestrated

Compose runs containers on **one** machine. Kubernetes runs them across a **cluster** and adds self-healing, autoscaling, and rolling updates as first-class features. This repo targets **local** clusters (Minikube / K3s / MicroK8s) — no cloud Kubernetes.

Manifest	Kubernetes objects	Compose equivalent
postgres-pv-pvc-secret.yaml	Namespace , Secret , PersistentVolumeClaim , StatefulSet , headless Service	the postgres service + its volume + env
deployments-and-services.yaml	4 × (Deployment + Service)	the 4 app services
gateway.yaml	Deployment + NodePort Service running calcul-nginx	the nginx service + its published port
ingress.yaml	Ingress	an alternative to gateway.yaml , using the cluster's shared ingress controller
hpa.yaml	2 × HorizontalPodAutoscaler	(no equivalent — Compose cannot autoscale)

Concept	Meaning
Deployment	“Keep N copies of this pod running; replace any that die; roll updates out gradually.”
Service	A stable internal name + IP for a changing set of pods.
StatefulSet	Like a Deployment but for things with identity and storage (databases). Postgres uses one.
Secret	Base64-wrapped config (⚠️ not encrypted at rest by default).
PersistentVolumeClaim	“I need 5 GB of disk”; the cluster provides it.
HorizontalPodAutoscaler	Watches CPU/memory and changes the replica count automatically (needs metrics-server).
readiness / liveness / startup probe	“Can it take traffic yet?” / “Is it wedged and needing a restart?” / “Give it time to boot before the other two apply.”

9. Correctness fixes made during cleanup

Area	Was	Now
history-service schema	Table only created by <code>db/init.sql</code> , so on Kubernetes (no init hook) the first write failed with <i>relation "calculation_history" does not exist</i> .	<code>init_db()</code> runs SQLAlchemy <code>create_all()</code> on startup, with a bounded retry while Postgres wakes up. Works everywhere.
DB connection pool	Fixed <code>pool_size=5</code> , <code>max_overflow=10</code> per worker — could exhaust Postgres' 100-connection limit across replicas.	Configurable via <code>DB_POOL_SIZE</code> / <code>DB_MAX_OVERFLOW</code> ; smaller documented defaults; <code>pool_recycle</code> added.
scientific-engine overflow	<code>math.exp(1000)</code> / huge <code>pow</code> raised <code>OverflowError</code> → generic HTTP 500.	Caught → HTTP 400 with a clear message. <code>factorial</code> input validated and capped at 170.
CI / Trivy	Used <code>aquasecurity/trivy-action@v0.36.0</code> (a Marketplace action from a family compromised in March 2026). Slack payload built by string-interpolating the commit message → a <code>"</code> broke the JSON.	Trivy runs from the pinned official image <code>aquasec/trivy:0.55.2</code> ; Slack payload built with <code>jq</code> (injection-safe); workflow given least-privilege <code>permissions:</code> and <code>concurrency:</code> .
Naming	Mixed: <code>history-service:latest</code> in k8s, <code>calcu-history-service</code> in Cl. <code>gateway.yaml</code> carried a byte-for-byte copy of <code>nginx.conf</code> in a <code>ConfigMap</code> .	Everything is <code>calcu-<service>.gateway.yaml</code> runs the real <code>calcu-nginx</code> image — one source of truth for routing.
Compose exposure	Postgres published on <code>0.0.0.0:5432</code> ; credentials hard-coded.	Not published at all on a server (localhost-only in local dev); credentials from <code>.env</code> .
Line endings	No <code>.gitattributes</code> — a Windows checkout could ship CRLF shell scripts that fail on Linux.	<code>.gitattributes</code> forces LF on scripts, Dockerfiles, YAML, the <code>systemd</code> unit.

Part II — Manual DevOps Runbook (“What You Should Do”)

Every exact command and every by-hand step to build, test, run, and operate `calcu` — locally and on a fresh AWS EC2 instance.

Legend: 🙋 = a human must do this by hand (web console / decision). 💻 = run this command exactly.

A. Things only a human can do

These cannot be scripted from inside the repo. Do them once.

A1. Accounts 🙋

Account	Needed for	Notes
AWS	the EC2 instance	Free-tier is enough (<code>t3.micro</code> / <code>t2.micro</code>).
Docker Hub	CI pushing images	Create an access token (Account Settings → Security). Never use your password.
GitHub	source + CI	The repo must be pushed here for GitHub Actions to run.
Slack (optional)	pipeline notifications	Create an Incoming Webhook (api.slack.com → Your Apps).
Jenkins host (optional)	CD	Any machine that can reach your Kubernetes cluster and has Docker + <code>kubectl</code> .

A2. Create the EC2 instance 🙋

AWS Console → EC2 → Launch instance:

Field	Value
Name	<code>calcu</code>
AMI	Ubuntu Server 24.04 LTS (x86_64)
Instance type	<code>t3.small</code> (2 GB RAM). <code>t3.micro</code> (1 GB) works but is tight with Postgres.
Key pair	Create or pick one. Download the <code>.pem</code> file — you need it to SSH in.
Network	default VPC, Auto-assign public IP = Enable
Firewall (security group)	create new, name it <code>calcu-sg</code> — rules in A3
Storage	20 GiB gp3

Launch. Note the **Public IPv4 address** once it is running.

A3. Security group rules 🙌

Edit `calcu-sg` → Inbound rules:

Type	Protocol	Port	Source	Why
SSH	TCP	22	My IP	So you can log in. Never <code>0.0.0.0/0</code> .
HTTP	TCP	80	<code>0.0.0.0/0</code> (and <code>:::/0</code>)	So the public can reach the app.

Do **not** open 5432 (Postgres), 5001–5003 (the engines), or 8080. Outbound: leave the default “all traffic”.

A4. IAM — do you need a role? 🙌

For this project as written (images built **on the instance** from the Git checkout): **no IAM role is required**. Attach an instance role later **only** if you switch to:

You want to...	Attach a role allowing...
Pull images from Amazon ECR instead of Docker Hub	<code>AmazonEC2ContainerRegistryReadOnly</code>
Read <code>POSTGRES_PASSWORD</code> from SSM Parameter Store	<code>ssm:GetParameter</code> on your parameter ARN
Ship logs to CloudWatch	<code>CloudWatchAgentServerPolicy</code>

Create via IAM → Roles → *Create role* → AWS service: *EC2* → attach policy → then EC2 → *Actions* → *Security* → *Modify IAM role*.

A5. (Optional) DNS 🙌

Point an `A` record at the instance’s public IP if you have a domain. TLS/HTTPS is out of scope here — add it later with Caddy, or an AWS Application Load Balancer + ACM certificate in front of the instance.

B. Run it locally from scratch

Assumes Ubuntu/Debian or WSL2. macOS: use Docker Desktop and skip B1.

B1. Install Docker Engine + Compose 🖥️

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable"
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
sudo usermod -aG docker "$USER"
newgrp docker      # or log out/in
```

Verify:

```
docker --version
docker compose version
docker run --rm hello-world
```

B2. Get the code

```
git clone <your-repo-url> calcu
cd calcu
```

B3. Configure

```
cp .env.example .env
# Local defaults are fine. To use a different host port:
# sed -i 's/^HTTP_PORT=.* HTTP_PORT=9090/' .env
```

B4. Build & start

```
docker compose build
docker compose up -d
docker compose ps          # wait until every service is "running"/"healthy"
```

B5. Verify

```
curl -fsS -X POST http://localhost:8080/api/scientific \
  -H 'Content-Type: application/json' -d '{"operation":"sqrt","value":16}'
# {"operation":"sqrt","result":4.0,"value":16.0}

curl -fsS -X POST http://localhost:8080/api/financial \
  -H 'Content-Type: application/json' \
  -d '{"operation":"emi","principal":100000,"annual_rate":10,"tenure_months":12}'

curl -fsS http://localhost:8080/api/history

docker compose exec scientific-engine curl -fsS http://localhost:5001/health
docker compose exec financial-engine curl -fsS http://localhost:5002/health
docker compose exec history-service   curl -fsS http://localhost:5003/health
```

Open **<http://localhost:8080>** in a browser.

B6. Logs / stop

```
docker compose logs -f          # all services, follow
docker compose logs -f history-service # one service
docker compose down             # stop & remove containers (keeps DB volume)
docker compose down -v         # ...and delete the database volume
```

B7. Run the unit tests

Needs Python 3.11+ (`sudo apt-get install -y python3 python3-pip python3-venv`).

```
cd calcul
for svc in scientific-engine financial-engine history-service; do
  echo "=== $svc ==="
  ( cd "services/$svc" \
    && python3 -m venv .venv && . .venv/bin/activate \
    && pip install -q -r requirements.txt pytest \
    && pytest -v \
    && deactivate )
done
```

`history-service` tests use an in-memory SQLite database — no Postgres needed.

C. Deploy on AWS EC2 from scratch

Do section A first (instance + security group exist).

C1. Connect to the instance

```
chmod 400 ~/Downloads/calcu-key.pem      # your .pem, once
ssh -i ~/Downloads/calcu-key.pem ubuntu@<EC2_PUBLIC_IP>
```

Everything below runs **on the instance**.

C2. Provision the instance (once)

```
sudo apt-get update && sudo apt-get install -y git
sudo mkdir -p /opt/calcu && sudo chown "$USER:$USER" /opt/calcu
git clone <your-repo-url> /opt/calcu
cd /opt/calcu
chmod +x deploy/*.sh
./deploy/ec2-provision.sh
newgrp docker      # apply docker group now (or log out and back in)
docker compose version
```

You now have **two** firewalls — the AWS security group *and* `ufw` . If you ever expose another port, open it in **both** (`sudo ufw allow <port>/tcp` and the security group). To rely on the security group alone: `sudo ufw disable` .

C3. Configure for production

```
cd /opt/calcu
cp .env.example .env
PW=$(openssl rand -base64 24)
sed -i "s|^POSTGRES_PASSWORD=.+|POSTGRES_PASSWORD=${PW}|" .env
sed -i "s|^HTTP_PORT=.+|HTTP_PORT=80|" .env
grep -E '^(HTTP_PORT|POSTGRES_PASSWORD)= ' .env
```

C4. Deploy

```
cd /opt/calcu
ENV=prod ./deploy/deploy.sh
```

This runs `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`. First run takes a few minutes (building 5 images).

C5. Verify

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml ps
curl -fsS -X POST http://localhost/api/scientific \
  -H 'Content-Type: application/json' -d '{"operation":"log10","value":1000}'
curl -fsS http://localhost/api/history

# from your laptop:
curl -fsS http://<EC2_PUBLIC_IP>/api/history
```

Then open `http://<EC2_PUBLIC_IP>` in a browser.

C6. Make it start on boot (systemd)

```
sudo cp /opt/calcu/deploy/calcu.service /etc/systemd/system/calcu.service
sudo systemctl daemon-reload
sudo systemctl enable --now calcu.service
sudo systemctl status calcu --no-pager
```

Now the stack comes back automatically after `sudo reboot`. Manage it with:

```
sudo systemctl restart calcu
sudo systemctl stop calcu
journalctl -u calcu -f
```

If you use systemd, let it own the lifecycle — do not also run `./deploy/deploy.sh` by hand. To ship an update, see E2.

D. Secrets & environment variables

D1. Local / EC2 — the `.env` file

- Lives at the repo root, **gitignored**, never committed.
- Created from `.env.example`. Compose loads it automatically.
- Contains: `HTTP_PORT`, `POSTGRES_USER/PASSWORD/DB`, pool sizing, `IMAGE_TAG`.
- Changing `POSTGRES_PASSWORD` after the volume exists does **not** change the password inside an already-initialised database — see E4.

D2. GitHub Actions secrets 🙌

Repo → Settings → Secrets and variables → Actions → New repository secret:

Name	Value
DOCKERHUB_USERNAME	your Docker Hub username
DOCKERHUB_TOKEN	the Docker Hub access token from A1
SLACK_WEBHOOK_URL	your Slack incoming-webhook URL

D3. Jenkins credentials 🙌

Manage Jenkins → Credentials → System → Global credentials → Add Credentials:

Kind	ID (must match exactly)	Value
Secret file	kubeconfig-calcu	your cluster's kubeconfig (~/.kube/config)
Secret text	slack-webhook-url	your Slack incoming-webhook URL

Then edit Jenkinsfile → DOCKERHUB_USERNAME = '<your-dockerhub-username>'. The Jenkins agent also needs kubectl and docker on its PATH.

D4. (Optional) pull secrets from AWS instead of .env 🙌💻

```
sudo apt-get install -y awscli
PW=$(aws ssm get-parameter --name /calcu/postgres_password --with-decryption \
  --query Parameter.Value --output text --region <your-region>)
sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=${PW}|" /opt/calcu/.env
```

E. Day-2 operations

E1. Inspect a running stack 💻

```
cd /opt/calcu
CF="-f docker-compose.yml -f docker-compose.prod.yml" # drop for local

docker compose $CF ps # container status + health
docker compose $CF logs -f --tail=100 # follow logs
docker stats --no-stream # live CPU/RAM per container
docker compose $CF exec postgres psql -U calcu_user -d calcu_db -c '\dt'
```

E2. Ship a new version (EC2)

If NOT using systemd:

```
cd /opt/calcu
git pull
ENV=prod ./deploy/deploy.sh
```

If using **systemd** (the unit rebuilds on start):

```
cd /opt/calcu
git pull
sudo systemctl restart calcu
journalctl -u calcu -f
```

E3. Roll back (Compose)

Compose has no built-in rollback. Pin to a previous commit:

```
cd /opt/calcu
git log --oneline -n 5
git checkout <previous-good-sha>
ENV=prod ./deploy/deploy.sh      # or: sudo systemctl restart calcu
```

(The Kubernetes path *does* auto-roll-back — see `Jenkinsfile` `post { failure } .`)

E4. Rotate the database password

```
cd /opt/calcu
CF="-f docker-compose.yml -f docker-compose.prod.yml"
NEW=$(openssl rand -base64 24)

# 1. change it inside the running database
docker compose $CF exec postgres \
  psql -U calcu_user -d calcu_db -c "ALTER USER calcu_user WITH PASSWORD '${NEW}';"

# 2. update .env so the app uses the new one
sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=${NEW}|" .env

# 3. recreate the app containers (NOT postgres) to pick it up
docker compose $CF up -d --no-deps --force-recreate history-service
```

E5. Back up / restore the history database

```
cd /opt/calcu
CF="-f docker-compose.yml -f docker-compose.prod.yml"

# Backup
docker compose $CF exec -T postgres \
  pg_dump -U calcu_user -d calcu_db | gzip > "calcu-$(date +%F).sql.gz"

# Restore (into an empty DB)
gunzip -c calcu-2026-01-01.sql.gz | \
  docker compose $CF exec -T postgres psql -U calcu_user -d calcu_db
```

E6. Free up disk

```
docker image prune -f          # dangling images
docker system df              # what's using space
docker system prune -a --volumes # AGGRESSIVE: removes unused images+volumes - check first
```

E7. Health & troubleshooting

Symptom	Command	Likely cause / fix
Site will not load from laptop	<code>curl -v http://<IP>/</code>	Security group missing port 80; or <code>HTTP_PORT</code> not 80 in <code>.env</code>
<code>history-service</code> unhealthy	<code>docker compose \$CF logs history-service</code>	Postgres still starting — it retries ~30s; if persistent, check <code>DATABASE_URL</code> and the <code>postgres</code> container
“Expected JSON but got text/html” in UI	—	You hit the wrong port — use the gateway (<code>:80</code> prod, <code>:8080</code> local), not a backend
Postgres won’t start, “directory not empty”	<code>docker compose \$CF logs postgres</code>	Corrupted volume — <code>docker volume rm calcu-postgres-data</code> (destroys data)
Out of memory on <code>t3.micro</code>	<code>free -m</code> , <code>docker stats</code>	Add a 2 GB swapfile (<code>fallocate</code> → <code>mkswap</code> → <code>swapon</code> → add to <code>/etc/fstab</code>)
Need a shell in a container	<code>docker compose \$CF exec <service> sh</code>	—

F. Teardown

F1. Stop the app, keep data

```
cd /opt/calcu
sudo systemctl disable --now calcu 2>/dev/null || true
docker compose -f docker-compose.yml -f docker-compose.prod.yml down
```

F2. Remove everything on the instance

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml down -v --rmi local
docker system prune -af --volumes
sudo rm -rf /opt/calcu
sudo rm -f /etc/systemd/system/calcu.service && sudo systemctl daemon-reload
```

F3. Delete AWS resources 🙌

EC2 Console → Instances → select `calcu` → **Instance state** → **Terminate**. Then delete the `calcu-sg` security group, the key pair (optional), and any IAM role you created.

Appendix — one-page cheat sheet

```
# ----- LOCAL -----
cp .env.example .env
docker compose up -d --build
docker compose ps
# http://localhost:8080
docker compose logs -f
docker compose down [-v]

# ----- EC2 (first time) -----
ssh -i key.pem ubuntu@<IP>
sudo mkdir -p /opt/calcu && sudo chown $USER /opt/calcu
git clone <repo> /opt/calcu && cd /opt/calcu
chmod +x deploy/*.sh && ./deploy/ec2-provision.sh && newgrp docker
cp .env.example .env # set HTTP_PORT=80 + POSTGRES_PASSWORD
ENV=prod ./deploy/deploy.sh
sudo cp deploy/calcu.service /etc/systemd/system/ && sudo systemctl enable --now calcu

# ----- EC2 (update) -----
cd /opt/calcu && git pull && sudo systemctl restart calcu

# ----- TESTS -----
cd services/<svc> && python3 -m venv .venv && . .venv/bin/activate \
&& pip install -r requirements.txt pytest && pytest -v
```

Generated for the `calcu` project. The canonical source is `DEVOPS_GUIDE.md` and `RUNBOOK.md` in the repository — if they diverge, the Markdown files win.